

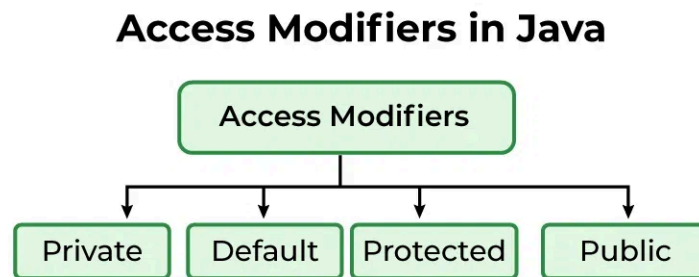
Access Modifiers in Java

In Java, **access modifiers** are essential tools that define how the members of a class, like **variables**, **methods**, and even the **class** itself, can be accessed from other parts of our program. They are an important part of building secure and modular code when designing large applications. In this article, we will explore each modifier with examples to demonstrate its impact on Java development.

Types of Access Modifiers

There are 4 types of access modifiers available in Java:

1. Default - No keyword required
2. Private
3. Protected
4. Public



Algorithm to Use Access Modifier in Java

Here's a basic algorithm for using access modifiers in Java:

1. **Define a class:** Create a class to represent the object you want to manage.
2. **Define instance variables:** Inside the class, define variables for the data you want to manage.
3. **Set an access modifier:**
 - Use private for variables only accessible within the class.
 - Use protected for variables accessible within the class and its subclasses.
 - Use public for variables accessible from anywhere.
4. **Use getter and setter methods:** To access or modify variables, use getter (accessor) and setter (mutator) methods, even for public variables, to maintain encapsulation.

1. Default Access Modifier

When no access modifier is specified for a class, method, or data member, it is said to have the default access modifier by default. This means only classes within the same package can access it.

Example 1: Demonstrating Default Access Modifier Within the Same Package

In this example, we will create two packages and the classes in the packages will be having the default access modifiers and we will try to access a class from one package from a class of the second package.

```
// default access modifier
package p1;

// Class Demo is having Default access modifier
class Demo
{
    void display()
    {
        System.out.println("Hello World!");
    }
}

// error while using class from different package with default modifier
package p2;
import p1.*; // importing package p1

// This class is having default access modifier
class DemoNew
{
    public static void main(String args[])
    {
        // Accessing class Demo from package p1
        Demo d = new Demo();
        d.display();
    }
}
```

Explanation: In this example, the program will show the compile-time error when we try to access a default modifier class from a different package.

2. Private Access Modifier

The private access modifier is specified using the keyword private. The methods or data members declared as private are accessible only within the class in which they are declared.

- Any other class of the same package will not be able to access these members.
- Top-level classes or interfaces cannot be declared as private because, private means "only visible within the enclosing class". Protected means "only visible within the enclosing class and any subclasses".

These modifiers in terms of application to classes, apply only to nested classes and not on top-level classes.

Example: In this example, we will create two classes A and B within the same package p1. We will declare a method in class A as private and try to access this method from class B and see the result.

```
// Error while using class from different package with
// private access modifier
package p1;
```

```
// Class A
class A {
    private void display() {
        System.out.println("GeeksforGeeks");
    }
}
```

```
// Class B
class B {
    public static void main(String args[]) {
        A obj = new A();

        // Trying to access private method
        // of another class
        obj.display();
    }
}
```

Explanation: The above code will show a compile-time error when trying to access a private method from class B, even within the same package.

3. Protected Access Modifier

The protected access modifier is specified using the keyword `protected`. The methods or data members declared as protected are accessible within the same package or subclasses in different packages.

Example 1: In this example, we will create two packages p1 and p2. Class A in p1 is made public, to access it in p2. The method `display` in class A is protected and class B is inherited from class A and this protected method is then accessed by creating an object of class B.

```
// protected access modifier
package p1;

// Class A
public class A
{
    protected void display()
    {
        System.out.println("Welcome to MIT WPU");
    }
}
```

So, it demonstrates that a protected method is accessible within the same package.

Example 2: In this example, we will create two packages, p1 and p2. Class A in p1 has a protected method display. Class B in p2 extends A and accesses the protected method through [inheritance](#) by creating an object of class B.

```
// protected modifier
package p2;
// importing all classes in package p1
import p1.*; // Class B is subclass of A
class B extends A
{
    public static void main(String args[])
    {
        B obj = new B();
        obj.display();
    }
}
```

Explanation: The above example demonstrates that a protected method is accessible in a subclass from a different package using inheritance.

4. Public Access Modifier

The public access modifier is specified using the keyword public.

- The public access modifier has the widest scope among all other access modifiers.
- Classes, methods, or data members that are declared as public are accessible from everywhere in the program. There is no restriction on the scope of public data members.

Example 1: Here, the code shows that a public method is accessible within the same package.

```
// public modifier
package p1;
public class A
{
    public void display()
    {
        System.out.println("Welcome to MITWPU");
    }
}
```

Example 2: Here, the example shows that a public method is accessible across packages.

```
// public access modifier
package p2;
import p1.*;
class B
{
    public static void main(String args[])
    {
        A obj = new A();
        obj.display();
    }
}
```

Comparison Table of Access Modifiers in Java

Context	Default	Private	Protected	Public
Same Class	Yes	Yes	Yes	Yes
Same Package Subclass	Yes	No	Yes	Yes
Same Package Non-Subclass	Yes	No	Yes	Yes
Different Package Subclass	No	No	Yes	Yes
Different Package Non-Subclass	No	No	No	Yes

When to Use Each Access Modifier in Real-World Projects

- **Private:** This is used for encapsulating sensitive data and internal helper methods that should not be accessed outside the class.
- **Example:** Private fields in a model class with getter and setter methods.
- **Default (Package-Private):** This is suitable for classes and methods that should only be accessible within the same package, often used in package-scoped utilities or helper classes.
- **Protected:** This is ideal for methods and fields that should be accessible within the same package and subclasses, commonly used in inheritance-based designs like framework extensions.
- **Public:** This is used for classes, methods, or fields meant to be accessible from anywhere, such as API endpoints, service classes, or utility methods shared across different parts of an application.